

Table 1: The benefits of using Docker for microservices

Benefit	Description
Portability	Docker containers run consistently across environments—development, testing, staging, and production.
Isolation	Each microservice runs in its own container, reducing conflicts and improving security.
Lightweight and fast	Containers are faster to start and use fewer resources compared to virtual machines.
Simplified dependency management	Docker bundles an app with all its dependencies, ensuring consistent behaviour across systems.
Scalability	Services can be scaled independently using orchestration tools like Kubernetes or Docker Swarm.
Improved DevOps workflow	Seamless integration with CI/CD pipelines enables automated testing, building, and deployment.
Version control and rollbacks	Docker images can be versioned, allowing easy rollbacks to previous versions of a microservice.
Ecosystem and community support	A vast repository of prebuilt images (Docker Hub) and an active community accelerate development.

Table 2: Key design principles for scalable microservices

Principle	Description
Single responsibility	Each service should own one specific business function (e.g., payment, authentication). This ensures clear ownership, simplified logic, and easier scaling.
Loose coupling	Services should interact via well-defined APIs without tightly binding to each other's internal logic or data models. This allows teams to update, scale, or redeploy services independently.
High cohesion	Internally, each service should handle related tasks. This ensures that services are logically grouped and easier to manage.
Stateless design	Stateless services don't retain user or session information between requests. This makes it easier to scale horizontally by adding more service instances behind a load balancer.
Service autonomy	Services should manage their own database and configurations to avoid hidden dependencies and bottlenecks.
Asynchronous communication	Use message brokers (e.g., Kafka, RabbitMQ) for event-driven architecture. This helps decouple services, handle high volumes gracefully, and reduce the risk of cascading failures.

Retry logic and circuit breakers: To stop failures in one service from impacting others, use tools such as Hystrix or Resilience4j to apply resilience patterns.

Scalable services manage spikes in traffic by:

- Using Kubernetes or Docker Swarm to automatically scale containers.
- Monitoring response times, throughput, and bottlenecks with metrics and alerts.
- Ensuring that requests are idempotent, particularly for operations that can be automatically retried.

To prevent service interruptions, new service versions should be deployed while maintaining backward compatibility.

Containerising microservices with Docker

The next stage after designing and implementing

microservices is to package and run them consistently across environments. Docker becomes crucial at this point. By combining the application code, runtime, libraries, and dependencies into a small, portable container, Docker allows developers to containerise every microservice.

Microservices are used in containers to:

- Maintain uniformity throughout the environments of development, testing, and production.
- Isolate the environment of each service, making dependency management easier.
- Speed up deployment and scaling because containers are less resource-intensive than virtual machines and start up faster.
- Permit changes to be pushed separately for every service by enabling micro-deployments.